xgboost_timewindow 环境启用说明

2026 年 8 月 19 日

目 录

第 1 章 环境启用总览 **1**

1.1 工作区与脚本入口 1

1.2 整体处理框图 1

1.3 处理流程与脚本对应关系 1

第 2 章 训练环境搭建与 Python 依赖 **2**

2.1 环境基本要求 2

2.1.1 硬件配置要求 2

2.1.2 操作系统与基础软件 2

2.2 基于 Conda package 传输的环境搭建 2

2.2.1 联网环境中创建训练环境 2

2.2.2 打包 Conda 环境 3

2.2.3 目标机器解压与启用 3

2.2.4 Conda 环境验证 4

2.3 基于 pip 的备用搭建方式 4

2.4 依赖组成 4

2.5 验证脚本 5

2.6 推荐环境选择 5

第 3 章 FSDBAPI 与 Verdi/FsdbReader 环境 **7**

3.1 FSDBAPI 目录 7

3.2 编译与动态库 7

3.3 FSDB 前处理的可选激活脚本 7

第 4 章 处理脚本说明 **9**

4.1 FSDB 导出、标签提取与降维前处理 9

4.2 Step1：time-window 样本构造 10

4.3 Step2：代理特征筛选 11

4.4 代理有效性验证 13

4.5 验证入口与 time-window 主入口 14

第 5 章 典型启动方式 **16**

5.1 小规模验证 16

5.2 主训练：MCP 代理筛选 16

5.3 主训练：XGBoost 代理筛选 16

5.4 FSDB 前处理 17

第 6 章 关键环境变量 **18**

6.1 基础路径变量 18

6.2 训练输入输出变量 19

6.3 训练控制变量 19

6.4 代理筛选与 XGBoost 变量 20

第 7 章 启用前检查清单 **21**

7.1 训练链路 21

7.2 FSDB 链路 21

第 1 章　环境启用总览

1.1　工作区与脚本入口

本文统一使用 {env_dir} 表示 xgboost_timewindow 包根目录；迁移至其他机器时，应将该变量替换为目标机器上的 xgboost_timewindow 目录。

包根目录定义如下：

```
{env_dir}
```

脚本默认将 PROJECT_ROOT 定位到包目录的上一级，即：

```
{env_dir}/..
```

1.2　整体处理框图

流程入口分为 FSDB 前处理、time-window 主入口和快速验证三类；框图展示数据处理、代理筛选与代理有效性验证之间的关系，具体入口脚本在后续章节中说明。

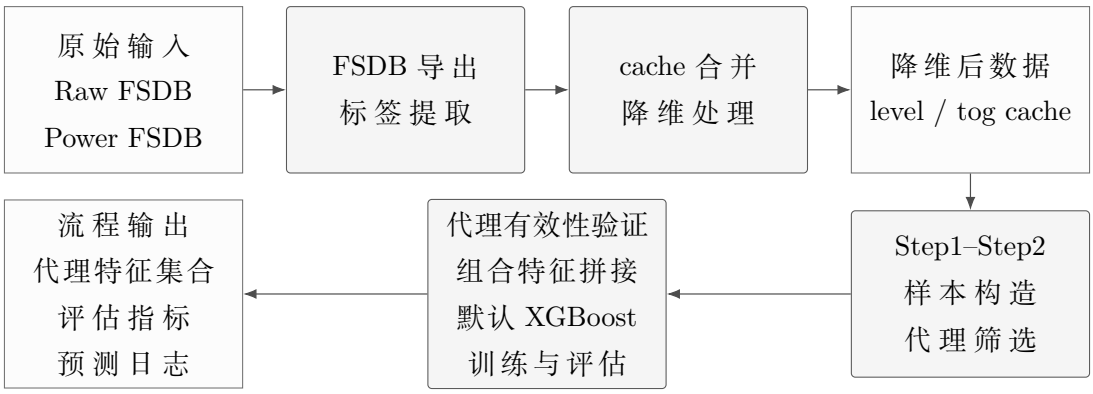


图 1-1　xgboost_timewindow 环境启用与处理流程概览

1.3　处理流程与脚本对应关系

后续说明按照图 1-1 的数据流展开。前处理阶段首先由 cmd/run_fsdb_pipeline.csh 调用 FSDBAPI 工具完成 FSDB 导出、标签提取、cache 合并与降维。time-window 主入口完成 Step1 样本构造与 Step2 代理筛选后，调用 cmd/run_timewindow_step2_validation.csh 对各 offset/type 组合的代理特征执行独立 XGBoost 验证，最终生成代理特征集合、评估指标和预测日志。快速验证入口 cmd/validate_timewindow_training_pro.csh 使用较小配置检查环境与数据路径是否可用。

各阶段脚本路径均相对于 {env_dir}。各处理阶段的脚本功能、接口参数与示例命令见第 4 章；典型启动方式见第 5 章，关键环境变量说明见第 6 章。

第 2 章　训练环境搭建与 Python 依赖

2.1　环境基本要求

2.1.1　硬件配置要求

训练环境的硬件规模应根据 FSDB 数量、降维后特征维度、采样行数和 XGBoost 候选特征数确定。参考配置如下：

- **CPU：**建议使用多核 Linux 服务器；Step1、Step2 与主 pipeline 可按 offset/type 或 top-K 拆分并行提交。
- **内存：**建议不低于 600 GB；当 TARGET_COLS、SAMPLE_ROWS_TOTAL 或 SAMPLE_ROWS_PER_FILE 增大时，应同步提高内存预算。
- **存储：**建议预留 TB 级空间；原始 FSDB、降维 cache、Step1 中间特征、XGBoost buffer、训练日志和验证结果会同时占用空间。
- **GPU：**当前脚本默认使用 CPU 版 XGBoost hist tree method；GPU 不是必需项。

2.1.2　操作系统与基础软件

本包面向 Linux-64 环境使用。训练链路主要依赖 Python、NumPy/SciPy 稀疏矩阵、scikit-learn、XGBoost、Matplotlib 与可选的 skglm；FSDB 前处理链路额外依赖 C++17 编译器、make、Verdi/FsdbReader 头文件与动态库。为保证复现一致性，建议使用独立 Conda 环境固定 Python 包版本，并在提交长作业前记录 PYTHON、输入数据目录和输出目录。

2.2　基于 Conda package 传输的环境搭建

本项目推荐在可联网机器上完成 Conda 环境创建和依赖解析，再将环境打包后传输至目标运行机器解压使用。该方式可以避免目标机器无法访问外网、镜像源不可用或在线求解依赖版本不一致等问题，也便于在多台运行服务器之间复用同一套训练环境。

2.2.1　联网环境中创建训练环境

在可联网机器上安装 Miniconda 或站点维护的 Conda 发行版，并配置稳定的软件源。若使用 conda-forge，建议启用 strict channel priority：

```
conda config --add channels conda-forge
conda config --set channel_priority strict
conda --version
```

随后创建独立训练环境，例如 pro。若已有环境规格文件，可使用如下方式创建：

```
conda create --name pro --file requirement.txt
```

若使用 YAML 形式维护环境，可改用：

```
conda env create -f requirement.yaml
```

环境创建完成后，在联网机器上先执行一次依赖验证，确认核心包能够正常导入：

```
conda run -n pro python - <<'PY'
import numpy, scipy, sklearn, pandas, xgboost, matplotlib
print("deps ok", "xgboost", xgboost.__version__, "matplotlib", matplotlib.__version__)
try:
    import skglm
    print("skglm ok")
except ImportError:
    print("skglm missing: Step2 MCP will use variance fallback")
PY
```

2.2.2 打包 Conda 环境

建议使用 conda-pack 将已验证环境打包为可传输文件。若联网机器尚未安装 conda-pack，可在 base 环境或独立工具环境中安装：

```
conda install -c conda-forge conda-pack
```

生成环境包：

```
conda pack -n pro -o pro-conda-env.tar.gz
```

打包文件 pro-conda-env.tar.gz 即为需要传输到目标机器的 Conda package。传输方式可使用 scp、共享文件系统或站点已有文件同步工具。

2.2.3 目标机器解压与启用

目标机器上建议将环境解压到与本文默认路径一致的位置，使训练脚本可以直接使用以下解释器：

```
PYTHON={env_dir}/conda/envs/pro/bin/python
```

解压命令如下：

```
mkdir -p {env_dir}/conda/envs/pro
tar -xzf pro-conda-env.tar.gz -C {env_dir}/conda/envs/pro
{env_dir}/conda/envs/pro/bin/conda-unpack
```

conda-unpack 会修正环境包中的前缀路径。完成后，批处理作业通常不需要依赖交互式 condaactivate；只要 PYTHON 指向解压后的解释器，Python 包加载路径即由该环境提供。

2.2.4　Conda 环境验证

解压完成后，应在目标机器上确认解释器路径与核心包版本：

```
{env_dir}/conda/envs/pro/bin/python - <<'PY'
import sys
import numpy, scipy, sklearn, pandas, xgboost, matplotlib
print(sys.executable)
print("numpy", numpy.__version__)
print("scipy", scipy.__version__)
print("sklearn", sklearn.__version__)
print("xgboost", xgboost.__version__)
print("matplotlib", matplotlib.__version__)
try:
    import skglm
    print("skglm ok")
except ImportError:
    print("skglm missing: Step2 MCP will use variance fallback")
PY
```

若目标机器上的解压目录与默认路径不同，应在启动训练前显式覆盖 PYTHON，并在作业日志中记录实际解释器路径。

2.3　基于 pip 的备用搭建方式

当目标机器无法直接使用 Conda 时，可使用 Python 内置 venv 创建隔离环境。该方式适合调试和小规模验证；生产作业仍建议使用已验证 Conda 环境。

```
cd {env_dir}
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

安装完成后使用同样的导入检查命令确认 numpy、scipy、sklearn、xgboost、matplotlib 和可选 skglm 是否可用。若需要迁移 pip 环境，可导出精确版本：

```
pip freeze > requirements_lock.txt
```

2.4　依赖组成

根据训练、降维和验证脚本的导入关系，Python 运行环境至少需要包含以下依赖。当前版本来自实际 pro 环境，解释器为 {env_dir}/conda/envs/pro/bin/python，Python 版本为 3.8.17。

依赖	当前版本	使用位置
numpy	1.24.4	标签、采样索引、稀疏矩阵元数据与数值计算。

scipy	1.10.1	scipy.sparse 的 load_npz、save_npz、hstack、vstack、csr_matrix。
pandas	2.0.3	降维元数据、MCP 评估和结果表处理。
scikit-learn	1.3.0	Ridge 回归及基础机器学习组件。
xgboost	2.0.1	Step2 XGBoost proxy 与代理有效性验证。
joblib	1.3.2	MCP 相关并行或模型辅助。
matplotlib	3.7.0	代理有效性验证中的指标与预测结果绘图。
skglm	0.3.1dev	可选推荐依赖；存在时 Step2 MCP 使用 MCPRegression，缺失时回退至 sparse variance top-K。

2.5 验证脚本

建议首先运行验证脚本完成最小闭环检查。该脚本会输出 Python 路径、检查核心依赖，并基于已有降维 cache 执行 Step1、Step2 与代理有效性验证的小规模配置：

```
cd {env_dir}
RUN_TAG=pro_timewindow_train_verify \
PYTHON={env_dir}/conda/envs/pro/bin/python \
csh cmd/validate_timewindow_training_pro.csh
```

验证脚本可按硬件名称覆盖以下输入路径。无显式传入 HW 时，time-window 主入口按 gemmini 作为默认硬件/数据集名称；迁移到其他数据集时，应同步替换 ${HW}、RUN_TAG 与降维输出目录。

```
HW=gemmini
RUN_TAG=<reduction_run_tag>
SMOKE_ROOT={env_dir}/../output/5fsdb_reduction_train_smoke
RUN_NAME=${HW}_coremark_5fsdb_${RUN_TAG}
SOURCE_RUN=${SMOKE_ROOT}/${RUN_NAME}
REDUCED_DIR=${SOURCE_RUN}/02_reduction/${HW}_5fsdb
LEVEL_DIR=${REDUCED_DIR}/level
TOG_DIR=${REDUCED_DIR}/tog
LABEL_DIR={env_dir}/../output/timewindow_test/power_labels
```

当输入路径发生迁移时，可在启动命令前覆盖对应变量。

2.6 推荐环境选择

训练链路建议使用已验证的 Conda 环境解释器：

```
PYTHON={env_dir}/conda/envs/pro/bin/python
```

cmd/timewindow_common.csh 和 cmd/run_timewindow_pipeline.csh 在未显式设置 PYTHON 时均使用上述路径。因此，一般无需执行 condaactivate；当 PYTHON 指向该解释器时，Python 依赖将由 pro 环境提供。

建议在启动命令前显式设置 PYTHON，以便在作业日志中记录实际使用的解释器：

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
csh cmd/validate_timewindow_training_pro.csh
```

第 3 章　FSDBAPI 与 Verdi/FsdbReader 环境

3.1　FSDBAPI 目录

FSDB 相关工具位于：

```
{env_dir}/src/fsdbapi
```

Makefile 优先使用工程内置的 FsdbReader：

```
{env_dir}/src/fsdbapi/third_party/verdi/FsdbReader
```

若 third_party/verdi/FsdbReader/ffrAPI.h 存在，编译过程将使用该目录；否则回退至外部 Verdi：

```
VERDI_HOME={env_dir}/../../../tools/verdi/verdi/R-2020.12-SP2
FSDB_READER_DIR=${VERDI_HOME}/share/FsdbReader
FSDB_READER_PLATFORM=linux64
```

3.2　编译与动态库

FSDBAPI 的 C++ 导出器通过 g++ 和 C++17 编译，链接 libnffr.so、libnsys.so 等 FsdbReader 库。Makefile 已设置 rpath：

```
LDFLAGS += -Wl,-rpath,$(FSDB_LIB_DIR)
```

因此，当二进制在原构建环境中运行时，通常无需额外设置 LD_LIBRARY_PATH。若移动二进制或切换 Verdi/FsdbReader 路径，应优先重新编译：

```
cd {env_dir}/src/fsdbapi
make clean
make VERDI_HOME=/new/path/to/verdi/R-2020.12-SP2
```

临时排查时可设置以下运行时变量：

```
setenv FSDB_LIB /new/path/to/verdi/R-2020.12-SP2/share/FsdbReader/linux64
setenv LD_LIBRARY_PATH ${FSDB_LIB}:${LD_LIBRARY_PATH}
```

3.3　FSDB 前处理的可选激活脚本

cmd/run_fsdb_pipeline.csh 在初始化后会检查包根目录是否存在 activate.csh：

```
if ( -f ${TOOLS_ROOT}/activate.csh ) then
    source ${TOOLS_ROOT}/activate.csh
endif
```

FSDB 前处理阶段可通过新增或维护以下文件：

```
{env_dir}/activate.csh
```

该文件用于集中设置站点相关变量，例如 PATH、LD_LIBRARY_PATH、license 变量、PYTHON 或 VERDI_HOME。该文件不是必需项；缺失时脚本沿用当前 shell 环境。

第 4 章　处理脚本说明

本章按照图 1-1 的处理顺序展开脚本功能、实现思路、接口参数和示例命令。FSDB 前处理与 time-window 主流程建议优先通过 cmd/run_fsdb_pipeline.csh 和 cmd/run_timewindow_pipeline.csh 启动；Step2 代理筛选完成后，由独立验证脚本使用默认 XGBoost 参数检查代理结果。当需要定位问题、复用中间结果或重跑单步时，可使用本章列出的单步命令。

4.1　FSDB 导出、标签提取与降维前处理

功能描述：该阶段将原始 RTL FSDB 与 power FSDB 转换为训练链路可读取的数据。入口脚本读取 workload、FSDB 路径、采样起止时间与 power signal 配置，调用 FSDBAPI 导出 level/tog cache，随后提取功耗标签，并执行 x/z 清理、全 0/全 1 常量列剔除、重复列处理和可选相似列降维。输出目录通常包含 level/、tog/、*_meta.json 与 *.namefile.txt。

处理机制：该阶段的核心在于先把不适合建模或对目标没有区分能力的信号列从特征矩阵中清理出去，再进入代理特征抽取与模型训练。FSDB 中的 x/z/X/Z 表示未知态或高阻态，不能稳定解释为 0/1 活动；当原始多 bit 信号中出现这类状态时，脚本会将该原始信号拆分得到的 bit 列统一标记为无效列，避免未知态被误当作有效翻转或电平状态参与训练。全 0 信号在全部样本中没有电平变化或翻转事件，全 1 信号虽然取值固定但没有样本间差异，二者均属于零方差特征；这类列对监督模型的损失下降、特征重要性排序和代理筛选没有有效贡献，保留下来只会增加稀疏矩阵的列规模。

在常量列清理之后，流程继续处理重复列和可选相似列降维。重复或高度相似的信号通常携带相同或近似相同的信息，在 MCP、Ridge 或 XGBoost proxy 等后续代理抽取阶段会反复占用候选特征名额，并放大矩阵拼接、列索引映射和 importance 统计的时间与内存开销。因此，dimentionality_reduction_csv.py 基于 cache manifest 对列集合进行清洗和记录，只保留具有有效变化、可区分样本且映射关系明确的候选列。清理结果通过 meta.json、final_cols.txt 与 original_headers.txt 保存，使后续 Step1、Step2 以及验证阶段可以直接复用降维后的稀疏 level/tog cache，而不需要在训练阶段重新读取 FSDB 或再次处理无效信号。

脚本路径：

```
cmd/run_fsdb_pipeline.csh
src/fsdbapi/build/fsdb_to_tog_cache
src/fsdbapi/build/fsdb_to_power_label
src/fsdbapi/bin/merge_tog_cache_manifests.py
src/reduction/dimentionality_reduction_csv.py
```

接口参数：

- fsdb_list：输入 workload、RTL FSDB 路径、采样起点和终点。
- power_list：输入 workload、power FSDB 路径和功耗信号名。

- STEP：FSDB tag 采样步长，用于对齐 RTL 特征与 power label。
- RAW_COL：FSDB 导出阶段的原始列块大小。
- MAX_MEM：merge 与降维阶段的内存预算，例如 28GB。
- TARGET_COLS：降维后的目标特征数量。
- MAX_CYCLES_PER_FILE：cache 输出的最大行分片长度，0 表示自动策略。
- SIGNAL_PREFIX：可选信号层级过滤前缀，用于限制导出信号范围。
- BSUB_QUEUE、BSUB_EXTRA_ARGS：集群提交队列与额外资源参数。

示例命令：

```
cd {env_dir}
csh cmd/run_fsdb_pipeline.csh gemmini
```

若 FsdbReader 路径发生变化，应先重新编译 FSDBAPI：

```
cd {env_dir}/src/fsdbapi
make clean
make VERDI_HOME=/new/path/to/verdi/R-2020.12-SP2
```

4.2　Step1：time-window 样本构造

功能描述：Step1 根据降维后的 level/tog cache 与 power label 构造 time-window 样本。对于每个 offset/type 组合，脚本生成训练/测试 feature 与 label 文件列表，并写出 timewindow_manifest.json。当主 pipeline 启用全局采样时，step1_make_global_sample_index.py 会先基于 power label 构造跨 case 的采样索引，随后由 step1_prepare_timewindow.py 复用该索引生成一致的训练输入。

处理机制：time-window 建模的核心是将目标时刻的功耗标签与其前若干 offset 的 RTL 活动状态配对。Step1 只做样本组织和文件列表生成，不在该阶段训练模型；这样可以把昂贵的矩阵构造与后续筛选、拼接、训练解耦。按 offset/type 拆分输出能够让 Step2 对每个时间偏移和数据类型独立筛选代理特征，也便于主入口按组合并行提交作业。

脚本路径：

```
src/timewindow/step1_make_global_sample_index.py
src/timewindow/step1_prepare_timewindow.py
cmd/run_timewindow_step1_prepare.csh
```

接口参数：

- --level-dir、--tog-dir、--label-dir：降维 level/tog cache 与 power label 输入目录。
- --output-dir：Step1 输出目录，通常为 ${OUTPUT_ROOT}/step1_shifted。
- --offsets：time-window 偏移列表，默认 12,11,10,9,8,7,6。

- --data-types：输入类型，默认 tog,level。
- --boundary-discard：边界丢弃长度，缺省时取最大 offset。
- --test-count：作为测试集的 case 数量。
- --stage：降维阶段名，默认 after_one。
- --sample-rows-per-file、--sample-rows-total：按文件或全局控制采样行数，0 表示保留全部可用样本。
- --sample-bins、--sample-min-per-bin、--sample-seed：功耗分层采样的 bin 数、每 bin 最小样本数和随机种子。
- --jobs：按 offset/type 并行处理的 worker 数。
- --include-stems：仅处理指定 case stem，空值表示处理全部有效 case。
- --sample-index-manifest：全局采样索引 manifest。
- --compress-output、--skip-existing：控制 npz 压缩和已有中间结果复用。

示例命令：

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python
${PYTHON} src/timewindow/step1_prepare_timewindow.py \
  --level-dir ${LEVEL_DIR} \
  --tog-dir ${TOG_DIR} \
  --label-dir ${LABEL_DIR} \
  --output-dir ${OUTPUT_ROOT}/step1_shifted \
  --offsets 12,11,10,9,8,7,6 \
  --data-types tog,level \
  --test-count 1 \
  --stage after_one \
  --compress-output 0 \
  --skip-existing 1
```

4.3 Step2：**代理特征筛选**

功能描述：Step2 按 Step1 manifest 中的 offset/type 组合执行代理筛选。MCP 路径优先使用 skglm.MCPRegression，若环境中缺少 skglm 则回退为 sparse variance top-K；XGBoost proxy 路径训练轻量 XGBoost 模型并按 gain importance 选择候选列。两种路径均输出每个组合的 selected index、可选 feature name 和汇总 manifest。

处理机制：降维后特征数量仍可能很大，直接把所有 offset/type 特征拼接后训练会显著增加内存与搜索成本。Step2 先在每个组合内部做局部代理筛选：MCP 通过稀疏正则化产生非零系数集合，随后用 Ridge 在测试集上选择较优 batch；XGBoost proxy 则利用浅层

小模型的 gain importance 估计候选列贡献。两种方法都输出统一格式的列索引和 manifest，因此代理有效性验证可以直接读取任一筛选结果。

脚本路径：

```
src/timewindow/step2_mcp_per_offset.py
src/timewindow/step2_xgb_per_offset.py
cmd/run_timewindow_step2_mcp.csh
cmd/run_timewindow_step2_xgb.csh
cmd/run_timewindow_pipeline_xgb_proxy.csh
```

接口参数：

- --manifest：Step1 生成的 timewindow_manifest.json。
- --namefile：降维阶段生成的 signal name 文件，用于输出可追踪特征名。
- --output-dir：Step2 输出目录。
- --batch：分批处理特征列的 batch 大小。
- --top-k：每个 offset/type 组合保留的代理特征数量。
- --jobs：按组合并行的 worker 数。
- --include-combos：仅处理指定组合，例如 tog_offset12。
- MCP 参数：--alpha、--gamma、--alpha-l2。
- XGBoost proxy 参数：--rounds、--eta、--max-depth、--subsample、--colsample-bytree、--min-child-weight、--reg-lambda、--nthread、--seed。

示例命令：

```
cd {env_dir}
${PYTHON} src/timewindow/step2_mcp_per_offset.py \
  --manifest ${OUTPUT_ROOT}/step1_shifted/timewindow_manifest.json \
  --namefile ${NAMEFILE} \
  --alpha 50.0 \
  --gamma 3.0 \
  --alpha-l2 8.0 \
  --top-k 200 \
  --batch 4 \
  --jobs 1 \
  --output-dir ${OUTPUT_ROOT}/step2_mcp
```

```
cd {env_dir}
${PYTHON} src/timewindow/step2_xgb_per_offset.py \
  --manifest ${OUTPUT_ROOT}/step1_shifted/timewindow_manifest.json \
  --namefile ${NAMEFILE} \
```

```
--top-k 200 \
--rounds 50 \
--eta 0.08 \
--max-depth 4 \
--output-dir ${OUTPUT_ROOT}/step2_xgb_proxy
```

4.4 代理有效性验证

功能描述：该流程读取 Step1 生成的 time-window manifest 与 Step2 生成的代理筛选 manifest，按 offset/type 组合提取 Step2 选中的特征列，构建独立验证数据集，并使用脚本内置的默认参数训练一版 XGBoost 功耗预测模型。验证结果用于检查 selected indices 是否能够正确映射到 Step1 特征、不同组合的标签是否一致、训练与测试样本是否正确划分，以及代理特征是否能够支撑基本的功耗预测。

处理机制：pipeline_combo_dataset.py 读取每个组合的 train/test feature filelist 与 selected_indices_file，按相同 stem 校验标签后执行列选择和横向拼接。为限制内存占用，脚本按 STEP2_VALIDATION_CHUNK_SIZE 将稀疏矩阵保存为 X.npz、y.npy 和 m.npy 分块，并在 dataset.meta.json 中记录组合顺序、特征名称、文件索引和周期索引。xgb_power_v2_3.py 将 Step1 的 test stems 固定为测试集，其余 train stems 按随机种子 108 打乱后划分训练集与验证集，并通过 XGBoost DataIter 和 QuantileDMatrix 逐块加载数据，避免在 Python 侧合并完整稀疏矩阵。

默认模型训练 200 个 boosting rounds，使用 hist tree method、max_depth=6、max_leaves=64、learning_rate=0.05、subsample=0.8、colsample_bytree=0.01 和 colsample_bynode=0.01。训练结束后导出逐样本预测 CSV，xgb_power_eval_from_prediction_csv.py 分别计算 overall、split、file 和 file/split 粒度的 RMSE、NRMSE、MAPE、R^2 与误差分位数，并对同一文件中连续周期的真实功耗跳变和预测功耗跳变进行阈值统计。该过程读取但不修改 Step2 manifest，训练模型及评估结果均写入独立的验证输出目录。

处理脚本：

```
cmd/run_timewindow_step2_validation.csh
src/timewindow/step2_validation/pipeline_combo_dataset.py
src/timewindow/step2_validation/xgb_power_v2_3.py
src/timewindow/step2_validation/xgb_power_eval_from_prediction_csv.py
```

关键输入与参数：

- timewindow_manifest.json：提供组合顺序、train/test stems、特征文件列表、标签文件列表与采样周期信息。
- mcp_per_offset_manifest.json：提供每个 offset/type 组合的 selected_indices_file；MCP 与 XGBoost proxy 均使用该 manifest 接口。
- NAMEFILE：将被选列恢复为原始 RTL signal 名称，供 feature importance 与结果追踪使用。

- STEP2_VALIDATION_COMBOS：指定参与验证的组合，默认由 OFFSETS 与 DATA_TYPES 展开得到。
- STEP2_VALIDATION_CHUNK_SIZE：组合特征拼接时每个稀疏分块的行数，默认值为 200。
- RUN_STEP2_VALIDATION：控制主 pipeline 是否在 Step2 完成后执行验证，默认值为 1。

主要输出：

- dataset_cache/dataset.meta.json 与 dataset_cache/chunks/：组合代理特征的分块缓存及数据映射。
- legacy_xgb_validation/tile_reset_domain-bc6/result/：XGBoost 模型、树结构、特征名称映射与 gain importance。
- legacy_xgb_validation/tile_reset_domain-bc6/eval/：逐样本预测 CSV、点误差指标、连续周期跳变明细、阈值汇总、直方图与 evaluation_summary.json。

示例命令：

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
OUTPUT_ROOT=/path/to/output/timewindow_train \
STEP2_VALIDATION_COMBOS=tog_offset12,level_offset12 \
STEP2_VALIDATION_CHUNK_SIZE=200 \
csh cmd/run_timewindow_step2_validation.csh gemmini
```

若需要验证指定的 Step2 manifest，可将其作为第二个位置参数传入：

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
csh cmd/run_timewindow_step2_validation.csh gemmini \
  /path/to/mcp_per_offset_manifest.json
```

4.5　验证入口与 time-window 主入口

功能描述：验证入口固定采用小规模配置，用于检查环境、依赖、输入路径以及代理筛选到有效性验证的最小闭环；time-window 主入口负责设置目录、提交 dsub 任务、维护 complete/failed 状态文件，并根据命令行阶段参数运行代理筛选主链路或局部验证流程。

处理机制：验证入口把解释器、依赖导入、输入 cache 和各 Step 的最小闭环放在同一个脚本中，适合在迁移环境后快速暴露路径或 Python 包问题。time-window 主入口则把目录推导、默认变量、任务拆分、状态文件和日志路径集中管理，使单步脚本可以保持相对简单。由于各阶段以 manifest、分块缓存或 CSV 作为边界文件，主入口可以检测已有产物并支持局部重跑或续跑。

脚本路径：

```
cmd/validate_timewindow_training_pro.csh
cmd/run_timewindow_pipeline.csh
cmd/run_timewindow_pipeline_xgb_proxy.csh
cmd/timewindow_common.csh
```

接口参数：

- 命令行第一个参数 HW：硬件或数据集名称，默认示例为 gemmini。
- 命令行第二个参数：阶段选择；本文所述流程使用 all 或 prep。
- PYTHON、PROJECT_ROOT、OUTPUT_ROOT：解释器、工作区根目录和训练输出目录。
- LEVEL_DIR、TOG_DIR、LABEL_DIR、NAMEFILE：训练输入数据与信号名映射。
- PROXY_SELECTOR：代理筛选路径，mcp、xgb 或 xgboost。
- RUN_STEP2_VALIDATION、STEP2_VALIDATION_COMBOS、STEP2_VALIDATION_CHUNK_SIZE：代理有效性验证开关、组合范围与缓存分块大小。

示例命令：

```
cd {env_dir}
RUN_TAG=pro_timewindow_train_verify \
PYTHON={env_dir}/conda/envs/pro/bin/python \
csh cmd/validate_timewindow_training_pro.csh
```

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
PROJECT_ROOT={env_dir}/.. \
LEVEL_DIR=/path/to/reduced/level \
TOG_DIR=/path/to/reduced/tog \
LABEL_DIR=/path/to/power_labels \
NAMEFILE=/path/to/reduced/level/after_one.namefile.txt \
OUTPUT_ROOT=/path/to/output/timewindow_train \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

若需要使用 XGBoost proxy，可直接调用封装入口：

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
XGB_PROXY_TOP_K=200 \
XGB_PROXY_ROUNDS=50 \
csh cmd/run_timewindow_pipeline_xgb_proxy.csh gemmini all
```

第 5 章　典型启动方式

本章给出验证、主训练和 FSDB 前处理的常用启动命令。示例命令采用 bash/sh 风格的一次性内联变量覆盖；若在 csh/tcsh 会话中手动执行，可先使用 setenv 设置同名变量。各变量的默认来源、覆盖方式和迁移注意事项见第 6 章。

5.1　小规模验证

```
cd {env_dir}
RUN_TAG=pro_timewindow_train_verify \
PYTHON={env_dir}/conda/envs/pro/bin/python \
csh cmd/validate_timewindow_training_pro.csh
```

为降低内存占用和运行时间，可在验证阶段启用采样：

```
cd {env_dir}
SAMPLE_ROWS_PER_FILE=128 \
SAMPLE_BINS=16 \
SAMPLE_MIN_PER_BIN=2 \
PYTHON={env_dir}/conda/envs/pro/bin/python \
csh cmd/validate_timewindow_training_pro.csh
```

5.2　主训练：MCP 代理筛选

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
PROJECT_ROOT={env_dir}/.. \
LEVEL_DIR=/path/to/reduced/level \
TOG_DIR=/path/to/reduced/tog \
LABEL_DIR=/path/to/power_labels \
NAMEFILE=/path/to/reduced/level/after_one.namefile.txt \
OUTPUT_ROOT=/path/to/output/timewindow_train \
OFFSETS=12,11,10,9,8,7,6 \
DATA_TYPES=tog,level \
RUN_STEP2_VALIDATION=1 \
STEP2_VALIDATION_CHUNK_SIZE=200 \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

5.3　主训练：XGBoost 代理筛选

建议使用专用封装脚本：

```
cd {env_dir}
PYTHON={env_dir}/conda/envs/pro/bin/python \
XGB_PROXY_TOP_K=200 \
XGB_PROXY_ROUNDS=50 \
XGB_PROXY_MAX_DEPTH=4 \
XGB_PROXY_ETA=0.08 \
csh cmd/run_timewindow_pipeline_xgb_proxy.csh gemmini all
```

也可通过主入口变量直接启用：

```
PROXY_SELECTOR=xgb XGB_PROXY_TOP_K=200 \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

5.4 FSDB 前处理

FSDB 前处理入口默认沿用当前 shell 环境。若包根目录下存在可选的 activate.csh，脚本会优先读取该文件。运行前应确认 FSDBAPI 二进制和动态库配置：

```
cd {env_dir}/src/fsdbapi
make

cd {env_dir}
csh cmd/run_fsdb_pipeline.csh gemmini
```

若站点 Verdi/FsdbReader 不在默认位置，编译时覆盖：

```
cd {env_dir}/src/fsdbapi
make VERDI_HOME=/new/path/to/verdi/R-2020.12-SP2
```

第 6 章　关键环境变量

本章整理训练链路与 FSDB 前处理链路中需要重点关注的环境变量。脚本整体采用“调用环境优先、脚本默认值补齐”的方式：在启动命令前已经设置的变量通常会被保留；未设置的变量由入口脚本根据 {env_dir}、HW、STAGE 和 RUN_TAG 推导默认路径。

time-window 主入口 cmd/run_timewindow_pipeline.csh 会集中设置 PACKAGE_ROOT、PROJECT_ROOT、PYTHON、输入 cache 目录、输出目录、Step1–Step2 代理筛选参数以及代理有效性验证参数；单步脚本通过 cmd/timewindow_common.csh 复用同一套默认值。cmd/run_timewindow_pipeline_xgb_proxy.csh 会先设置 XGBoost proxy 相关变量，再调用 time-window 主入口。验证脚本 cmd/validate_timewindow_training_pro.csh 使用独立的小规模默认配置，用于快速检查解释器、依赖和输入数据是否可用。

FSDB 前处理入口 cmd/run_fsdb_pipeline.csh 主要设置原始 FSDB 目录、前处理输出目录、临时目录和 Verdi/FsdbReader 相关路径；站点级变量可集中写入 {env_dir}/activate.csh，由 FSDB pipeline 在启动时加载。若从 bash/sh 启动一次性运行，建议在命令前以内联变量形式覆盖；若处于 csh/tcsh 交互会话或批处理脚本中，可使用 setenv 固定本次 shell 中的变量。

```
PYTHON={env_dir}/conda/envs/pro/bin/python \
LEVEL_DIR=/path/to/reduced/level \
TOG_DIR=/path/to/reduced/tog \
LABEL_DIR=/path/to/power_labels \
OUTPUT_ROOT=/path/to/output/timewindow_train \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

下列变量表按照基础路径、训练输入输出、训练控制以及代理筛选/XGBoost 参数分组。迁移数据或复现实验时，应优先检查 PYTHON、PROJECT_ROOT、LEVEL_DIR、TOG_DIR、LABEL_DIR、NAMEFILE 和 OUTPUT_ROOT；其中输入目录与 NAMEFILE 必须来自同一批降维结果，输出目录应指向新的运行目录或明确可续跑的已有目录。

6.1　基础路径变量

变量	默认值或来源	说明
PACKAGE_ROOT	{env_dir}	脚本包根目录。
PROJECT_ROOT	PACKAGE_ROOT/..	工作区根目录，默认对应 {env_dir}/..。

TIMEWINDOW_SRC	PACKAGE_ROOT/src/timewindow	Step1–Step2 代理筛选与代理有效性验证脚本目录。
REDUCTION_BASE	PROJECT_ROOT/output 下的 xgboost_timewindow_fsdb_pipeline	FSDB 前处理和降维输出根目录。
OUTPUT_BASE	PROJECT_ROOT/output 下的 xgboost_timewindow_train	训练输出根目录。
WORK_DIR	PROJECT_ROOT/tmp 下的 xgboost_timewindow_train_work	主训练临时目录。

6.2 训练输入输出变量

变量	默认值或示例	说明
HW	命令第一个参数，默认 gemmini	硬件/数据集名称，用于拼接默认输入输出路径。
LEVEL_DIR	REDUCTION_BASE 下的 HW_reduced/level	降维后的 level cache 目录。
TOG_DIR	REDUCTION_BASE 下的 HW_reduced/tog	降维后的 tog cache 目录。
LABEL_DIR	REDUCTION_BASE 下的 HW_power	power label 的 .npy 文件目录。
STAGE	after_one	选择 namefile 与降维阶段。
NAMEFILE	LEVEL_DIR 下的 STAGE.namefile.txt	signal name 映射文件。
RUN_TAG	当前时间戳	本次输出标识。
OUTPUT_ROOT	OUTPUT_BASE 下的 HW_STAGE_RUN_TAG	本次训练结果目录。

6.3 训练控制变量

变量	默认值	说明
OFFSETS	12,11,10,9,8,7,6	time-window 偏移列表。
DATA_TYPES	tog,level	使用 tog、level 或二者。
TEST_COUNT	1	test case 数量。
SAMPLE_ROWS_PER_FILE	0	每个 case 采样行数；0 表示不按文件采样。
SAMPLE_ROWS_TOTAL	0	全局采样总行数；主 pipeline 支持。

SAMPLE_BINS	20	power 分层采样 bin 数。
SAMPLE_SEED	108	采样随机种子。
STEP1_COMPRESS_OUTPUT	0	Step1 输出 npz 是否压缩。
STEP1_SKIP_EXISTING	1	已有中间结果的复用开关。
STEP1_PARALLEL_DSUB	1	主 pipeline 中 Step1 是否拆分提交 dsub。
STEP2_PARALLEL_DSUB	1	主 pipeline 中 Step2 是否拆分提交 dsub。

6.4 代理筛选与 XGBoost 变量

变量	默认值	说明
PROXY_SELECTOR	mcp	Step2 筛选器；可设为 xgb 或 xgboost。
ALPHA_MCP	50.0	MCPRegression alpha。
GAMMA	3.0	MCPRegression gamma。
ALPHA_L2	8.0	Ridge refinement alpha。
MCP_TOP_K	200	无 skglm 时 variance fallback 的 top-K。
XGB_PROXY_TOP_K	200	XGBoost proxy 每个 offset/type 导出的列数。
XGB_PROXY_ROUNDS	50	Step2 小模型 boosting rounds。
XGB_PROXY_MAX_DEPTH	4	Step2 小模型最大深度。
XGB_PROXY_ETA	0.08	Step2 小模型学习率。
XGB_PROXY_SUBSAMPLE	0.9	Step2 小模型行采样率。
XGB_PROXY_COLSAMPLE	0.8	Step2 小模型列采样率。
XGB_PROXY_NTHREAD	1	Step2 小模型线程数。
RUN_STEP2_VALIDATION	1	Step2 完成后是否执行独立 XGBoost 验证。
STEP2_VALIDATION_COMBOS	由 OFFSETS/DATA_TYPES 生成	参与验证的 offset/type 组合及拼接顺序。
STEP2_VALIDATION_CHUNK_SIZE	200	验证数据组合拼接时每个稀疏分块的行数。

第 7 章　启用前检查清单

7.1　训练链路

提交训练前应检查以下项目：

1. PYTHON 是否指向 {env_dir}/conda/envs/pro/bin/python 或其他已安装依赖的解释器。
2. 依赖检查命令是否能成功导入 numpy/scipy/sklearn/pandas/xgboost/matplotlib。
3. 若使用 MCP，确认是否需要真实 skglm；缺少 skglm 时将使用 variance fallback。
4. LEVEL_DIR、TOG_DIR、LABEL_DIR、NAMEFILE 是否存在且属于同一批数据。
5. OUTPUT_ROOT 是否指向新的输出目录或可续跑目录；STEP1_SKIP_EXISTING=1 将复用已有中间产物。
6. 使用主 pipeline 时，确认 DSUB_CMD、DSUB_QUEUE、DSUB_EXTRA_ARGS 与当前集群环境匹配。

7.2　FSDB 链路

运行 FSDB 前处理前应检查以下项目：

1. {env_dir}/src/fsdbapi/build 目录下的 fsdb_to_tog_cache 与 fsdb_to_power_label 是否已完成编译。
2. FSDB_READER_DIR 或 VERDI_HOME 是否能定位到 ffrAPI.h 与 linux64/libnffr.so。
3. 若二进制无法加载动态库，应优先重新编译；临时排查时可设置 LD_LIBRARY_PATH。
4. cmd/run_fsdb_pipeline.csh 中的 fsdb_list、power_list、STEP、RAW_COL、SIGNAL_PREFIX 是否符合本次数据。
5. 若需要站点环境变量，放入 {env_dir}/activate.csh，由 FSDB pipeline 自动 source。